

Experiment No: 8

TITLE: To develop a program for Binary Search.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about BinarySearch.
- Develop program for BinarySearch.

THEORY:

Binary Search:

The necessary condition for this method is that all the elements should be sorted either in ascending order or descending order. Once the data is sorted it is not necessary to compare each and every record. The number of searches required in this method is reduced.

ALGORITHMS:

BINARY_SEARCH(K,N,X)

Given an array K, consisting of N elements in ascending order, this algorithm searches structure for a given element whose value is given by X. The variables LOW, MIDDLE, HIGH denote lower, middle and upper limits of search interval. This function returns an index of a vector element if search is successful and returns 0 otherwise.

- i. [Initialize]
 LOW = 1
 HIGH = N
- ii. [Performsearch]
 Repeat thru step 4 while LOW <= HIGH
- iii. [Obtain index of midpoint of interval]
 MIDDLE = (LOW+HIGH)/2
- iv. [COMPARE]
 If X < K[MIDDLE] then HIGH = MIDDLE - 1
 Else
 If X > K[MIDDLE]
 Then LOW = MIDDLE + 1
 Else Write („SUCCESSFUL SEARCH“)
 Return (MIDDLE)
- v. [UnsuccessfulSearch]
 Write („UNSUCCESSFUL SEARCH“)
 Return (0)

EXERCISE:

1. Write a program in C to implement the above Binary Search Algorithms.

DATE:

Aim: To develop a program for Binary Search.

Program:

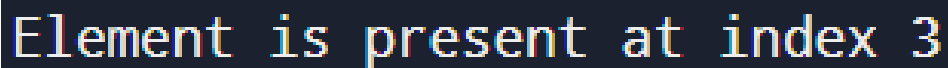
```
#include <stdio.h>

int binarySearch (int arr[], int size, int key) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key)
            return mid;
        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[] = {1, 5, 10, 15, 20};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 15;
    int result = binarySearch(arr, size, key);
    if (result != -1)
        printf("Element is present at index %d\n", result);
    else
        printf("Element is not present in array\n");
    return 0;
}
```

Output:

Here we searched for element '15' which is present as the output below.

A screenshot of a terminal window with a dark background. The text "Element is present at index 3" is displayed in a light blue, monospaced font. The text is centered horizontally and appears to be the output of a program.

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 9

TITLE: To develop a program for Bubble Sort.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about Bubble Sort.
- Develop program for Bubble Sort.
- Know about SelectionSort.
- Develop program for SelectionSort.

THEORY:

Sorting:

Sorting is systematic arrangement of the data. The systematic arrangement means based on some key the data should be arranged.

Different Techniques of sorting

1. BubbleSort
2. InsertionSort

ALGORITHMS:

BubbleSort

```
void BubbleSort(int x[], int n)
{
    int hold, j, pass;
    int switched = TRUE; //Outer loop controls the number of passes
    for(pass=0; pass<n-1 && switched == TRUE; pass++)
    {
        switched= FALSE; /*Initially no interchanges have been made on this
        pass*/
        //inner loop governs each individual pass
        for (j=0; j < n-pass-1; j++)
        {
            if (x[j]>x[j+1]) /*Interchange elements if not
            ordered*/
            {
                switched =
                TRUE; hold =
                x[j]; x[j] =
                x[j+1]; x[j+1] =
                hold;
            }
        }
    }
}
```

EXERCISE:

1. Write a program in C for sorting the elements by Bubble sort method.

DATE:

Aim: To develop a program for Bubble Sort.

Program:

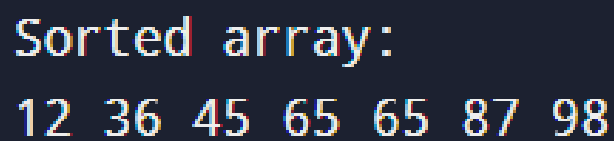
```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp;
            }
        }
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {98, 45, 65, 87, 12, 65, 36};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Output:



Sorted array:
12 36 45 65 65 87 98

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 10

TITLE: To develop a program for Insertion Sort.

OBJECTIVES:

After completing this experiment students will be able to...

- know about InsertionSort
- Develop program for InsertionSort.
- Know about MergeSort.
- Develop program for MergeSort.

THEORY:

Sorting:

Sorting is systematic arrangement of the data.

The systematic arrangement means based on some key the data should be arranged.

Different Techniques of sorting

1. InsertionSort
2. MergeSort

ALGORITHMS:

InsertionSort

```
void insertion_sort( int x[], int n)
```

```
{ int i, k,
```

```
y;
```

```
/*Initially x[0] may be thought of as a sorted file of one element. After each repetition of the following loop, the elements x[0] through x[k] are in order*/
```

```
for (k=1; k < n; k++)
```

```
{
```

```
//insert x[k] into the sorted file
```

```
y = x[k];
```

```
//Move down 1 position all elements greater than y
```

```
for(i = k-1; i >=0 && y < x[i]; i--)
```

```
x[i+1] = x[i];
```

```
//insert y at proper position
```

```
x[i+1] = y;
```

```
}
```

```
}
```

EXERCISE:

1. Write a program in C for sorting the elements by Insertion sort method.

DATE:

Aim: To develop a program for Insertion Sort & Merge Sort.

Program:

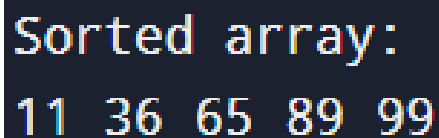
```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {65, 89, 36, 99, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    insertionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Output:



```
Sorted array:
11 36 65 89 99
```

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 11

TITLE: To develop a program for Quick sort.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about QuickSort.
- Develop program for QuickSort.
- Know about Radix Sort.
- Develop program for RadixSort.

THEORY:

Sorting:

Sorting is systematic arrangement of the data. The systematic arrangement means based on some key the data should be arranged.

Different Techniques of sorting

1. QuickSort
2. Radix Sort

ALGORITHMS:

quick Sort

```
#define MAXSTACK 100
struct bndtype
{
    int
    lb;
    int
    ub;
} newbnds;
//Stack is used by pop, push and empty functions
struct stack{
    int top;
    struct bndtype bounds[MAXSTACK];
};
void push(struct stack *s, struct bndtype *b)
{
    s->bounds[++s->top]=*b;
}
void popsub(struct stack *s, struct bndtype *b)
{
    *b=s->bounds[s->top--];
}
```

```

int empty(struct stack *s)
{ if(s->top<=-1)
return 1;
else return 0;
}
void partition(int x[], int lb, int ub, int *pj)
int a, down, temp, up;
a = x[lb];
up=ub;
down = lb;
while( down < up)
{
while (x[down] <= a && down < ub) down++;
while ( x[up] > a)up--;
if( down < up)
{
temp = x[down];
x[down] = x[up];
x[up] = temp;
} //end if
} //end while
x[lb] = x[up];
x[up] = a;
*pj = up; }
//end partition
void quicksort(int x[], int n)
{
int i, j; struct stack st;
st.top = -1;
newbnds.lb = 0;
newbnds.ub = n-1;
push( &st,
&newbnds);
/*repeat as long as there are any unsorted subarrays on the stack*/ while( !empty(&st))
{
popsub(&st, &newbnds);
while(newbnds.ub > newbnds.lb)
{
/*process nextsubarray*/
partition ( x, newbnds.lb, newbnds.ub, &j);
/*stack the larger subarray*/
if ( j-newbnds.lb > newbnds.ub-j)
{
/*stack lower subarray*/
i = newbnds.ub;
newbnds.ub = j-1;

```

```

push(&st, &newbnds);
/*process upper
subarray*/ newbnds.lb =
j+1;
newbnds.ub = i;
} else
{
/*Stack upper subarray*/
i = newbnds.lb;
newbnds.lb = j+1; push (
&st, &newbnds);
/*Process lower
subarray*/ newbnds.lb =
i;
newbnds.ub = j-1;\
} //end if
} //end while
} //endwhile
} //endquicksort

```

EXERCISE:

1. Write a program in C for sorting the elements by Quick sortmethod.

DATE:

Aim: To develop a program for Quick sort.

Program:

```
#include <stdio.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {8, 88, 58, 48, 68, 18};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("\nGiven array is\n");
    printArray(arr, n);
}
```

```

quickSort(arr, 0, n - 1);
printf("\nSorted array is \n");
printArray(arr, n);
return 0;
}

```

Output:

```

Given array is
8  88  58  48  68  18

Sorted array is
8  18  48  58  68  88

```

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 12

TITLE: To develop programs for various String Operations.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about various StringOperations
- Develop program for StringOperations.

THEORY:

String:

String is an array of characters.

Various operations to be performed on strings are

1. Find the stringlength.
2. Concat two strings.
3. Copying String.
4. Compare two strings.

ALGORITHMS:

i) Finding the length of the string

```
int strlen(char*s)
{ int length =
0; while(*s !=
„\0“)
{
length++
; s++; }
return (length);
}
```

ii) Copying a string into another string void

```
strcpy( char*t, char *s)
{
while( *s != „\0“)
{ *t =
*s;
s++;
t++; }
*t = „\0“;
}
```

iii) Concatenate two strings

```

char* strcat( char*s1, char *s2) {
char *result,*r;
r=result; while(
*s1 != '\0')
{
*result =
*s1; s1++;
result++; }
*result = ','; while(
*s2 != '\0')
{
*result =
*s2; s2++;
result++; }
*result='\0';
return r;
}

```

iv) Comparing two strings

```

int strcmp( char *s1, char *s2) {
while (*s1 == *s2)
{ if (*s1 ==
'\0') return 0;
s1++; s2++;
} return (-
1);
}

```

Exercise:

1. Write a program in C to find string length using the above algorithm.
2. Write a program in C to concat two strings.
3. Write a program in C to copy one string into another string.
4. Write a program in C to compare the two given strings.

DATE:

Aim: To develop programs for various String Operations.

Program:

String Length

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, World!";
    int length = strlen(str);
    printf("Length of the string: %d\n", length);
    return 0;
}
```

String Concat

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello, ";
    char str2[] = "World!";
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

String Copy

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello, World!";
    char dest[50];
    strcpy(dest, src);
    printf("Copied string: %s\n", dest);
    return 0;
}
```

String Compare

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "Hello";
    char str2[] = "World";
    int result = strcmp(str1, str2);
    if (result == 0) {
        printf("Strings are equal\n");
    }
    else if (result < 0) {
        printf("String 1 is less than String 2\n");
    }
    else {
        printf("String 1 is greater than String 2\n");
    }
    return 0;
}
```

Output:

Length of the string: 13

Copied string: Hello, World!

Concatenated string: Hello, World!

String 1 is less than String 2

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 13

TITLE: To develop a program for selection sort.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about selection sort
- Develop program and algorithm for selection sort.

THEORY:

Selection sort:

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

Algorithm

SELECTION SORT(arr, n)

Step 1: Repeat Steps 2 and 3 for i = 0 to n-1

Step 2: CALL SMALLEST(arr, i, n, pos)

Step 3: SWAP arr[i] with arr[pos]

[END OF LOOP]

Step 4: EXIT

SMALLEST (arr, i, n, pos)

Step 1: [INITIALIZE] SET SMALL = arr[i]

Step 2: [INITIALIZE] SET pos = i

Step 3: Repeat for j = i+1 to n if
(SMALL > arr[j])

 SET SMALL = arr[j]

SET pos = j

[END OF if]

[END OF LOOP]

Step 4: RETURN pos

Exercise:

1. Develop a program for selection sort.

DATE:

Aim: To develop a program for selection sort.

Program:

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {98, 12, 65, 35, 57};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Unsorted array: \n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Output:

```
Unsorted array:
98 12 65 35 57

Sorted array:
12 35 57 65 98
```

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date

Experiment No: 14

TITLE: To develop a program for Merge sort.

OBJECTIVES:

After completing this experiment students will be able to...

- Know about Merge sort.
- Develop program and algorithm for selection sort.

THEORY:

Merge sort:

Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort the elements. It is one of the most popular and efficient sorting algorithm. It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves. We have to define the merge() function to perform the merging.

Algorithm

```
MERGE_SORT(arr, beg, end)
```

```
if beg < end  set mid = (beg +  
end)/2
```

```
MERGE_SORT(arr, beg, mid)
```

```
MERGE_SORT(arr, mid + 1, end)  MERGE
```

```
(arr, beg, mid, end)
```

```
end of if
```

```
END MERGE_SORT
```

Exercise:

1. Develop a program for merge sort.

DATE:

Aim: To develop a program for Merge sort.

Program:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int leftArr[n1], rightArr[n2];
    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        }
        else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
}
```

```

while (j < n2) {
    arr[k] = rightArr[j];
    j++;
    k++;
}
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {78, 25, 65, 48, 15};
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    printf("Given array is \n");
    printArray(arr, arr_size);
    mergeSort(arr, 0, arr_size - 1);
    printf("\nSorted array is \n");
    printArray(arr, arr_size);
    return 0;
}

```

Output:

```
Given array is
78 25 65 48 15

Sorted array is
15 25 48 65 78
```

EVALUATION:

Involvement(4)	Understanding/Problem Solving(3)	Timely Completion(3)	Total(10)

Signature with date
